Trace Based Diagnosis and Repair of Component Faults in RAG Pipelines

UIC CS544 Machine Learning in Production | Hybrid project report

Group members. Sina Gogani Khiabani Verya Monjezi Ali Hoseinpour

Research problem. Retrieval augmented generation (RAG) services can produce wrong answers when evidence is lost or altered between retrieval and generation. We investigate whether observable execution traces can guide inexpensive repairs while preserving answers that were already correct.

Background. Published work identifies missing content and context-consolidation failures [1], outdated evidence [2], serving-window truncation [3], and configuration incompatibilities [4]. RAGChecker provides component diagnostics [5]; grounding and corrective-retrieval methods inform verification and repair [6-8]. These findings motivate a controlled study of runtime repair.

Goal and scope. We study retrieval, prompt assembly, serving, and index content in a local QA service. The contribution is a reproducible fault-injection harness and an empirical comparison of predefined trace checks, repair policies, monitoring signals, and regression tests.

Research questions

RQ1. Which repair policies recover answer accuracy under component faults while limiting clean-answer damage and computation?

RQ2. What incremental value do support checking, rechecking after repairs, and abstention provide?

RQ3. How quickly do trace and model-based signals detect faulty deployments, and how often do they raise false alarms?

RQ4. Can regression tests constructed from observed failures detect changes that break repair logic?

Working hypotheses. H1: Trace-guided repairs improve recovery and preserve more correct clean answers than generic self-checking or blanket repair under the same extra-call cap. H2: Additional support verification offers a smaller benefit while increasing computation and potentially unnecessary abstention.

General approach. Log retrieved passages, rendered text, version metadata, retrieval scores, and tokens sent and processed. Inject encoder mismatch, passage truncation, context overflow, stale versions, a missing answer document, and encoder mismatch plus truncation. Literature motivates the fault families; exact injections and severities are experimental choices. Hand-written checks trigger full rendering, window-aware context packing, newest-version selection, or keyword retrieval. Compare no repair, retry, self-check, blanket repair, trace checks only, and extended Diagnose variants, with at most two additional LLM calls. Repair decisions use observable traces; gold answers and fault labels are reserved for evaluation.

Evaluation benchmarks and metrics. Use a filtered HotpotQA subset [9], with 30 development and 150 held-out questions over 11,737 passages. Clean retrieval contains the annotated support. The prototype uses Llama-3.2-3B; a second generator is planned. Measure correctness, repair rate among initial failures, clean damage among initially correct answers, appropriate abstention, stale-value use, calls, tokens, and latency. Validate missing-evidence labels and human-audit answer equivalence. Use paired comparisons and question-level bootstrap confidence intervals. Evaluate detection delay, false alarms on independent clean traffic, and regression-test mutation detection, following production-testing principles [10].

Selected literature and project relevance

References 1-4 motivate the fault families; 5-8 cover diagnosis, verification, and repair; 9-10 support evaluation and production testing. Paper titles link to primary sources.

[1] Barnett, S., et al. (2024). Seven Failure Points When Engineering a Retrieval Augmented Generation System. CAIN 2024.
Fault taxonomy: motivates missing content, missed retrieval, and evidence lost during context consolidation.

[2] Ouyang, J., et al. (2025). HoH: A Dynamic Benchmark for Evaluating the Impact of Outdated Information on Retrieval-Augmented Generation. ACL 2025.
Stale evidence: studies outdated information coexisting with current sources; motivates our version-conflict condition.

[3] Musetoiu, D. (2026). RAGAL: A Frugal, Fully Local Retrieval-Augmented Assistant for Technical Support at a Government Agency. arXiv:2607.18756, preprint.
Serving fault: reports silent prompt truncation caused by an unset context-window parameter in a deployed assistant.

[4] Chen, L., et al. (2026). Configuring RAG Systems: What Breaks, Where Originates, and How to Fix. arXiv:2610.18567, preprint.
Configuration faults: documents RAG incompatibilities and cross-stage failures; informs our encoder-mismatch scenario.

[5] Ru, D., et al. (2024). RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. NeurIPS 2024, Datasets and Benchmarks.
Diagnostics: provides retrieval and generation metrics that inform our component-level evaluation.

[6] Tang, L., Laban, P., and Durrett, G. (2024). MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents. EMNLP 2024.
Support verification: supplies the grounding classifier used in our verification and repair-policy variants.

[7] Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884, preprint.
Corrective retrieval: motivates evaluating retrieved evidence and triggering alternative retrieval actions.

[8] Jiao, S., et al. (2026). Doctor-RAG: A Failure-Aware Repair Framework for Agentic Retrieval-Augmented Generation. arXiv:2604.00865, preprint.
Related repair method: provides a diagnosis-and-local-repair framework for comparison with our pipeline-focused approach.

[9] Yang, Z., et al. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. EMNLP 2018.
Evaluation benchmark: provides multi-hop questions, reference answers, and supporting-fact annotations.

[10] Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. IEEE Big Data 2017.
Production testing: guides the monitoring and regression-testing aspects of the project.